Singly Linked List – Insert At End –Space Complexity and Algorithm.

Program:

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
    }
```

```
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
void insertAtEnd(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    newNode->next = nullptr;
    if (*head == nullptr)
    {
        *head = newNode;
    }
    else
    {
        Node *current = *head;
        while (current->next != nullptr)
        {
            current = current->next;
        }
        current->next = newNode;
    }
}


int main()
{
   createEmptyList(&head);
    while (1)
    {
```

```
        cout << "1. Insert at the beginning" << endl;
        cout << "2. Insert at the end" << endl;

        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
      case 2:
      {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtEnd(&head, data);
            break;
      }
       default:
        {
            cout << "Invalid choice." << endl;
            break;
        }
        }
    }
  return 0;
}
```

Space Complexity

Input Space:

$a)\ Parameter - only\ view \quad \Theta(1)/O(1)/\Omega(1)$

$The\ input\ space\ refers\ to\ the\ space\ required\ to\ store\ the\ input\ data.$

$In\ this\ case, the\ input\ consists\ of\ a\ pointer\ to\ the\ head\ of\ the\ linked\ list$

$(Node\ **\ head)and\ an\ integer\ (int\ data).$

1. `Node **head` → one pointer-to-pointer
2. `int data` → one scalar

   The size of both parameters is fixed, therefore constant.

$The\ space\ required\ for\ these\ inputs\ is\ constant, i.e., O(1).$

$b)\ Full - data\ view \quad \Theta(n)$

This more common definition includes the entire data structure. If the linked list has **`n` nodes** before the insertion, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

Include everything reachable from the parameters when the call begins. *head may reference a singly–linked list containing n nodes → space already "in play" is proportional to n.

Auxiliary Space:

Auxiliary space refers to any additional space used by the algorithm, excluding the input space. In this function, the auxiliary space is primarily used for the creation of a new node.

The function allocates memory for a new node using malloc, which requires space proportional to the size of the Node structure.

This is also constant space, i.e., $O(1)$, because the size of the Node structure does not change with the size of the input.

What about the space used by temporary variables used by the program?

Temporary Variables:

- *Node * newNode: This is a pointer variable used to store the address of the newly allocated node. The space required for this pointer is constant, i.e., $O(1)$.*
- *Node * current: This is another pointer variable used to traverse the linked list to find the end. The space required for this pointer is also constant, i.e., $O(1)$.*

Overall Space Complexity:

Combining both the input space and the auxiliary space, the overall space complexity of the insertAtEnd function is $O(1)$. This is because the function only allocates a fixed amount of space for the new node, regardless of the size of the linked list.

Auxiliary-space complexity Θ(1)

Extra memory created or kept on this call's stack:

• Local pointer newNode (automatic variable) → constant

• Local pointer current → constant

• **malloc(sizeof(Node)) allocates exactly ONE node – often counted as "output space"; even if counted as auxiliary, it is a single fixed-size object ⇒ constant.**

• **The while-loop walks through the list but does not allocate new storage. Hence the routine's own workspace does not grow with n.**

Algorithm

$INSEND(START, ITEM)$

$\left((Node *)malloc(sizeof(Node))\ is\ availability\ of\ space\ for\ insertion\ of\ Data,\ hence\ ,lets\ name\ it\ AVAIL.\right)$

1. $[ALLOCATE\ NEW\ NODE]\ \ Set\ NEW := AVAIL.$

2. $[MEMORY\ ALLOCATED?]\ if\ NEW = NULL, then:$

$Write: ``\ Error: Memory\ Allocation\ Failed.\ ``\ and\ EXIT\ .$

$[End\ of\ If\ structure.]$

3. $Set\ INFO[NEW] := ITEM[Copies\ new\ data\ into\ new\ node].$

4. $Set\ LINK[NEW] := NULL\ [Sets\ the\ link\ of\ the\ new\ node\ to\ NULL, as\ this\ will\ be\ the\ last\ LINK\ of\ the\ Last\ Node\ ]$

5. $[When\ HEAD\ points\ to\ NULL\ i.e.\ \ LINK\ LIST\ IS\ EMPTY]$

$If\ START = NULL\ then:$

$Set\ \ START := NEW\ [START\ or\ HEAD\ points\ to\ NEW\ NODE's\ address]$

$[When\ HEAD\ doesn't\ points\ to\ NULL\ i.e. LINK\ LIST\ IS\ NOT\ EMPTY]$

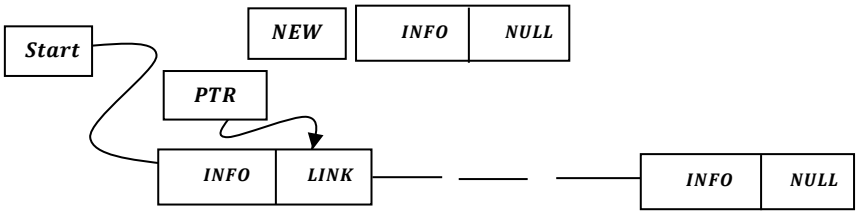
$ELSE$

$Set\ PTR := START\ [PTR\ is\ a\ pointer\ which\ is\ assigned\ to\ Head]$

$Repeat\ while\ LINK[PTR]\ \neq NULL : [Traverses\ list\ seeking\ last\ node.]$
$Set\ PTR := LINK[PTR]\ [Updates\ PTR.]$

$[END\ of\ Loop]$

$Set\ LINK[PTR] := NEW\ [Sets\ LINK\ of\ PTR\ to\ NEW\ node.]$

6. $Exit.$

DIAGRAM